

Använda exceptions

Det finns två skilda moment involverade då man använder sig av exceptions, en part som försöker exekvera en viss del av programmet och fånga upp fel, och en annan part som kan ge en exception. Då en exception aktiveras så *kastar* (throw) programmet en exception. Mottagaren har en skild sektion som är markerad som kritisk och exceptions som kastas från denna kritiska sektion fångas upp av en fångst-sektion. Vi ska se på ett konkret exempel på ett program som dividerar två tal som användaren matat in och som kastar en exception om man försöker dividera med 0:

```
#include <iostream>
#include <string>

float divide (float X, float Y) {
    // är täljaren 0?
    if ( Y == 0 ) {
        // jep, kasta en exception
        throw string ("Nämnaren måste vara != 0");
    }

    // värdena ok, räkna och returnera
    return X / Y;
}

int main () {
    float X, Y;

    // evig slinga
    while ( 1 ) {
        cout << "Ge täljaren: ";
        cin >> X;
        cout << "Ge nämnaren: ";
        cin >> Y;

        // försök skriva ut kvoten och fånga fel
        try {
            // kritisk sektion
            cout << X << "/" << Y << " = " << divide (X, Y) << endl;
        }
        catch (string Error) {
            // ett fel har fångats, skriv ut meddelandet
            cout << Error << endl;
        }
    }
}
```

Funktionen `divide()` kollar om nämnaren är 0, och i så fall kastas en exception. Detta görs med nyckelordet `throw`. Man kan kasta vilken datatyp som helst, allt från `int`, `string` till skräddarsydda klasser. I vårt fall vill vi endast förmedla ett meddelande om att vi dividerar med 0. anroparen vet även att detta är det enda fel som kan uppstå. Ifall flera olika fel kunde uppstå i våra beräkningar kunde vi kasta t.ex. en `enum` och sedan i anroparen kolla vilket fel som inträffade. I `main()` finns en slinga som läser in två tal och försöker dividera dessa. Den sektion där programmet förväntar sig en exception är i ett s.k. `try`-block. Efter detta kommer ett s.k. `catch`-block där man deklarerar de exceptions som man är förberedd på att ta emot. I programmet ovan tas en `string` emot. Man kan ha flera `catch`-block för att ta emot olika typer av exceptions. Allmänt ser en exception-deklaration ut

på följande sätt:

```
try {
    satser;
}
catch (datatyp1 variabel1) {
    satser;
}
catch (datatyp2 variabel2) {
    satser;
}
...
catch (...) {
    satser;
}
```

De olika `catch`-blocken måste ha olika datatyp som de försöker fånga. Man kan placera en sista "fånga alla" `catch` med datatypen `...` (tre punkter) för att fånga upp exceptions som inte tidigare fångats. Ifall där man har klasser som exceptions kommer det kastade undantaget att provas uppifrån och ned bland `catch`-satserna tills någon passar eller man når en `...`-sats. Det är även möjligt att nästa exceptions, d.v.s. ha ett nytt `try-catch`-block innanför t.ex. en `catch`-sektion.

Allokering och exceptions

Ett exempel på en operator som använder sig av exceptions är `new`, som kastar `bad_alloc` ifall den inte kunde allokera önskad mängd minne. Följande program allokerar minne tills det tar slut (ett bra exempel på faran med att läcka minne!):

```
#include <iostream>
#include <new>

int main () {
    int Allokerat = 0;

    while ( 1 ) {
        // försök allokera minne evigt
        try {
            new char [ 1000000 ];

            // åter en MB alokerat
            Allokerat++;
        }
        catch ( bad_alloc ) {
            // minnet är slut
            cout << "Minnet slut efter " << Allokerat << "Mb" << endl;
            return 1;
        }
    }
}
```

Allokeringen med `new` allokerar 1Mb med minne per gång och läcker det. Förr eller senare tar minnet slut i datorn och `new` misslyckas. Den kastar då `bad_alloc`. För att kunna använda denna exception måste man inkludera `<new>`, som innehåller definitionen på `bad_alloc`.

Exceptions och konstruktörer

Man kan även använda exceptions tillsammans med konstruktörer. Tittar man närmare på en

konstruktör märker man att den inte kan returnera något till anroparen. En konstruktör har inget sätt med vilket den kan förmedla att den inte kunde exekveras korrekt. Det finns flera olika suboptimala lösningar på problemet men exceptions torde vara den bästa lösningen. Följande situation kunde uppstå för en klass `File` som abstraherar en fil då man försöker skapa en instans genom att ge ett filnamn med:

```
try {
    // försök öppna filen
    File * InputData = new File ( Filename );

    // filen öppnades ok, läs data
    ...
}
catch ( FileNotFoundException E ) {
    // ingen sådan fil hittades
}
catch ( PermissionDenied E ) {
    // inga rättigheter att öppna/läsa filen
}
catch ( ... ) {
    // annat fel, kanske minnesallokering
}

// programmet fortsätter med filen läst eller felet hanterat
```

Återkasta en exception

Ett `catch`-block kan om det vill kasta vidare samma exception som det har fångat genom en tom `throw;` utan exception efter. På så vis skickas samma exception uppåt i anropshierarkin. Det gör det lätt för olika nivåer att reagera på ett felmeddelande utan större besvär. På så vis kan en `catch` på lägsta nivå försöka ordna upp felet innan den ger upp och skickar felet vidare till en högre nivå. Exemplet nedan visar hur man kan återkasta en exception.

```
#include <iostream>

// definiera en egen datatyp för exceptions
enum Exception { Problem, Illa, Hemska };

void thrower () {
    // något illa har hänt!
    throw Illa;
}

void catcher () {
    // försök anropa thrower och se vad som händer
    try {
        thrower ();
    }
    catch ( Exception E ) {
        // är det något vi kan hantera
        if ( E == Problem ) {
            // allt är ok, vi kan hantera problemet
            cout << "Ingen fara!" << endl;
        }
        else {
            // allvarligt problem
            throw;
        }
    }
}
```

```
void dummy () {  
    // ingen 'try' här inte  
    catcher ();  
}  
  
int main () {  
    // anropa 'dummy'  
    try {  
        dummy ();  
    }  
    catch ( Exception E ) {  
        // vilken typ av exception är det frågan om?  
        switch ( E ) {  
            case Problem : cout << "Ett litet problem" << endl; break;  
            case Illa :     cout << "Ett större problem" << endl; break;  
            case Hemsk :   cout << "Ett hemskt problem" << endl; break;  
        }  
    }  
}
```

Vi har funktionen `thrower()` som kastar en exception av typen `Exception`. Det första `catch`-blocket i `catcher()` tar emot undantaget och kontrollerar vilken typ av exception det är. Om det är en typ som den kan hantera (`Problem`) hanteras problemet internt, annars är det ett större problem och en tom `throw` återkastar undantaget vidare. I funktionen `dummy()` finns inget `try-catch`-block så exceptionen skickas vidare till `main()`. Man behöver alltså inte ha ett `try-catch`-block på varje nivå, utan man kan kasta en exception mellan många nivåer. I `main()` kollas sedan vilken typ av exception det var frågan om och denna hanteras på något sätt.

Notera den egna datatypen `Exception` som definierats för våra felmeddelanden. Den innehåller ingen text el.dyl., men i detta exempel är det endast typen av exception som är viktig.

[Förutgående](#)
Exceptions

[Hem](#)
[Upp](#)

[Nästa](#)
Ofångade exceptions